

ΤΕΧΝΗΤΗ ΝΟΗΜΟΣΥΝΗ II

Ομάδα 1: Airports



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ
UNIVERSITY OF PATRAS

Μέλη:

- **Ονοματεπώνυμο:** Γεώργιος Προυσιώτης
Αριθμός Μητρώου: 1101037
- **Ονοματεπώνυμο:** Θοδωρής Πετούρης
Αριθμός Μητρώου: 1083821
- **Ονοματεπώνυμο:** Πέππας Μιχαήλ
Αριθμός Μητρώου: 1101011
- **Ονοματεπώνυμο:** Παπαδήμα Αγγελική
Αριθμός Μητρώου: 1084820

GitHub page: <https://github.com/GeoPr04/airport-location-optimization>

Στάδιο Α – Απλή Χωροθέτηση Ενός Αεροδρομίου

Περιγραφή Προβλήματος:

Δίνονται επτά πόλεις με γνωστές καρτεσιανές συντεταγμένες (x_i, y_i) . Στόχος είναι να προσδιοριστεί η θέση ενός αεροδρομίου στις συντεταγμένες (X, Y) , έτσι ώστε να ελαχιστοποιείται το συνολικό κόστος μετακίνησης των κατοίκων προς αυτό.

Για τον υπολογισμό της απόστασης μεταξύ μιας πόλης i και του αεροδρομίου χρησιμοποιούμε την Ευκλείδεια μετρική:

$$d(i) = \sqrt{(x_i - X)^2 + (y_i - Y)^2}$$

Στο πρώτο στάδιο θεωρούμε ότι όλες οι πόλεις έχουν την ίδια βαρύτητα. Επομένως η αντικειμενική συνάρτηση κόστους ορίζεται ως:

$$C = \sum d(i)$$

δηλαδή το άθροισμα των αποστάσεων όλων των πόλεων από το αεροδρόμιο.

Μεθοδολογία:

Για την εύρεση της βέλτιστης θέσης χρησιμοποιήθηκε η μέθοδος **Monte Carlo Search**. Η μέθοδος αυτή δημιουργεί μεγάλο πλήθος τυχαίων υποψήφιων θέσεων μέσα στην περιοχή αναζήτησης και αξιολογεί κάθε θέση μέσω της αντικειμενικής συνάρτησης κόστους.

Για κάθε τυχαίο σημείο:

1. Υπολογίζονται οι αποστάσεις του από όλες τις πόλεις.
2. Υπολογίζεται το συνολικό κόστος ως άθροισμα των αποστάσεων.
3. Η καλύτερη λύση αποθηκεύεται όταν παρουσιάζει μικρότερο κόστος από όλες τις προηγούμενες.

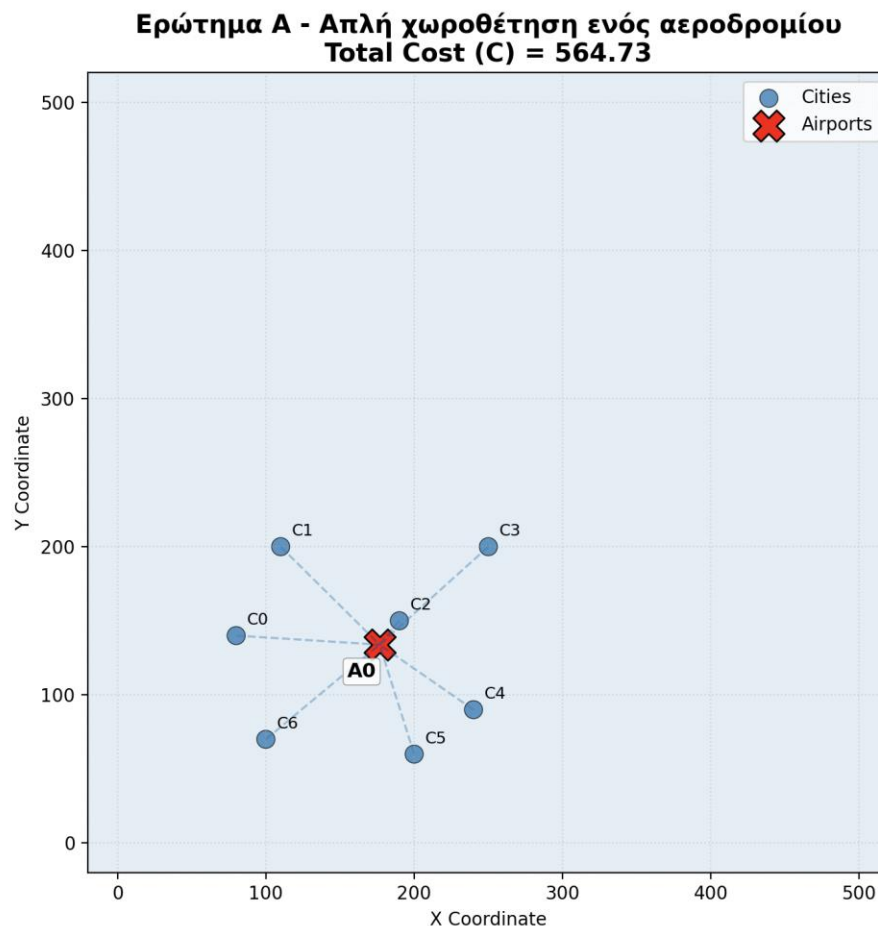
Η μέθοδος επιλέχθηκε επειδή είναι απλή στην υλοποίηση. Δηλαδή, δεν απαιτεί παραγώγους ή ειδικές μαθηματικές ιδιότητες της συνάρτησης κόστους και μπορεί να εφαρμοστεί εύκολα σε προβλήματα χωροθέτησης.

Υλοποίηση:

Στον κώδικα αποθηκεύονται οι συντεταγμένες των πόλεων σε έναν πίνακα NumPy. Η συνάρτηση κόστους υπολογίζει την Ευκλείδεια απόσταση κάθε πόλης από την υποψήφια θέση του αεροδρομίου και επιστρέφει το άθροισμά τους.

Μετά την ολοκλήρωση της διαδικασίας αναζήτησης εμφανίζεται η βέλτιστη θέση του αεροδρομίου καθώς και η αντίστοιχη τιμή κόστους. Παράλληλα δημιουργείται διάγραμμα στο οποίο παρουσιάζονται οι πόλεις, η τελική θέση του αεροδρομίου και οι συνδέσεις μεταξύ τους.

Διάγραμμα:



Σχολιασμός Αποτελεσμάτων:

Η βέλτιστη θέση που προέκυψε βρίσκεται κοντά στο γεωμετρικό κέντρο του συνόλου των πόλεων. Το οποίο είναι αναμενόμενο, καθώς όλες οι πόλεις συνεισφέρουν ισότιμα στη συνάρτηση κόστους και το αεροδρόμιο τείνει να τοποθετείται σε σημείο που εξισορροπεί τις αποστάσεις προς όλες.

Στάδιο B – Σταθμισμένη Χωροθέτηση Αεροδρομίου

Περιγραφή Προβλήματος:

Στο δεύτερο στάδιο λαμβάνεται υπόψη και ο πληθυσμός κάθε πόλης. Κάθε πόλη i διαθέτει πληθυσμό p_i και η απόσταση της από το αεροδρόμιο σταθμίζεται ανάλογα με το μέγεθός της.

Χρησιμοποιείται η γενική αντικειμενική συνάρτηση:

$$C = \sum p_i \cdot D_i$$

όπου:

$$D_i = \min d(i,j)$$

είναι η απόσταση της πόλης i από το πλησιέστερο αεροδρόμιο.

Επειδή στο συγκεκριμένο πρόβλημα υπάρχει μόνο ένα αεροδρόμιο ($m=1$), η παραπάνω σχέση απλοποιείται σε:

$$C = \sum p_i \cdot d(i)$$

Έτσι οι μεγαλύτερες πόλεις αποκτούν μεγαλύτερη επιρροή στη διαδικασία βελτιστοποίησης.

Μεθοδολογία:

Χρησιμοποιήθηκε η ίδια ακριβώς μέθοδος **Monte Carlo Search** όπως και στο Στάδιο Α.

Η μόνη διαφορά βρίσκεται στη συνάρτηση κόστους. Αντί να αθροίζονται απλώς οι αποστάσεις, κάθε απόσταση πολλαπλασιάζεται με τον πληθυσμό της αντίστοιχης πόλης:

$$\text{Cost} = \sum (\text{Population} \times \text{Distance})$$

Ενώ η διαδικασία αναζήτησης παραμένει ίδια:

1. Δημιουργία τυχαίας θέσης αεροδρομίου.
2. Υπολογισμός αποστάσεων από όλες τις πόλεις.
3. Πολλαπλασιασμός κάθε απόστασης με τον αντίστοιχο πληθυσμό.
4. Υπολογισμός συνολικού κόστους.
5. Διατήρηση της καλύτερης λύσης.

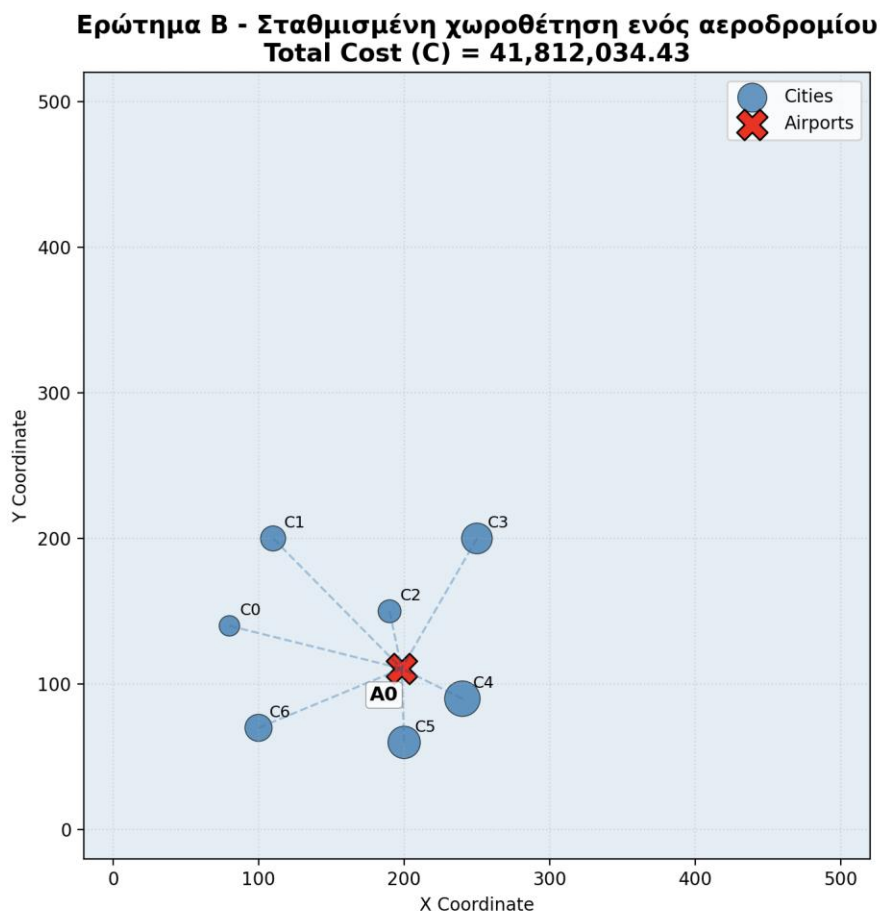
Άρα δεν χρειάστηκε αλλαγή της μεθόδου βελτιστοποίησης, αλλά μόνο τροποποίηση της αντικειμενικής συνάρτησης.

Υλοποίηση:

Στον κώδικα χρησιμοποιείται επιπλέον πίνακας populations που περιέχει τους πληθυσμούς των επτά πόλεων. Κατά τον υπολογισμό του κόστους κάθε απόσταση σταθμίζεται με τον αντίστοιχο πληθυσμό.

Στο τελικό διάγραμμα το μέγεθος κάθε πόλης απεικονίζεται αναλογικά με τον πληθυσμό της, ώστε να γίνεται εμφανής η σχετική σημασία κάθε αστικού κέντρου.

Διάγραμμα:



Σχολιασμός Αποτελεσμάτων:

Σε σύγκριση με το πρώτο στάδιο, η βέλτιστη θέση του αεροδρομίου μετατοπίστηκε προς τις πόλεις με τον μεγαλύτερο πληθυσμό. Αυτό είναι λογικό, καθώς η μείωση της απόστασης για μια μεγάλη πόλη επηρεάζει περισσότερο το συνολικό κόστος από την αντίστοιχη μείωση για μια μικρότερη πόλη.

ΣΤΑΔΙΟ Γ

Στο στάδιο αυτό εξετάστηκε το πρόβλημα τοποθέτησης δύο αεροδρομίων, όπου κάθε πόλη εξυπηρετείται από το πλησιέστερο αεροδρόμιο. Σε αντίθεση με το πρόβλημα ενός αεροδρομίου οι αποστάσεις στην συγκεκριμένη περίπτωση δεν υπολογίζονται από ένα κοινό σημείο αλλά από το κοντινότερο από τα δύο αεροδρόμια. Αυτό κάνει το πρόβλημα πιο σύνθετο καθώς η αντιστοίχιση των πόλεων στα αεροδρόμια αλλάζει δυναμικά κατά την διάρκεια της βελτιστοποίησης.

Για την επίλυση του προβλήματος χρησιμοποιήθηκε ο Genetic Algorithm επειδή η αντικειμενική συνάρτηση γίνεται μη γραμμική και δύσκολα επιλύεται αναλυτικά.

Κάθε chromosome αναπαριστά συντεταγμένες των δύο αεροδρομίων:

$$((\square_1, \square_1), (\square_2, \square_2))$$

Έπειτα υπολογίζεται το συνολικό σταθμισμένο κόστος μετακίνησης όλων των πόλεων προς το πλησιέστερο αεροδρόμιο. Για κάθε πόλη υπολογίζεται η ευκλείδεια απόσταση από τα δύο αεροδρόμια και επιλέγεται η μικρότερη:

$$\square = \sqrt{(\square_{\square} - \square)^2 + (\square_{\square} - \square)^2}$$

Η υπολογισμός του κόστους είναι:

$$\square = \sum \square_{\square} * \square(\square_{\square 1}, \square_{\square 2})$$

Όπου :

$\square_{\square}$: είναι ο πληθυσμός της πόλης i

$\square_{\square 1}, \square_{\square 2}$: οι αποστάσεις από το πρώτο και δεύτερο αεροδρόμιο αντίστοιχα.

Ο γενετικός αλγόριθμος ξεκινά με τυχαίες θέσεις αεροδρομίων και επαναληπτικά εφαρμόζει selection, crossover και mutation, ώστε να παράγει συνεχώς καλύτερες λύσεις. Μετά από αρκετές γενιές ο αλγόριθμος συγκλίνει σε θέσεις που ελαχιστοποιούν τη συνολική σταθμισμένη απόσταση.

Παρατηρείται ότι οι πόλεις χωρίζονται φυσικά σε δύο ομάδες. Άρα το βέλτιστο αποτέλεσμα τοποθετεί ένα αεροδρόμιο στην αριστερή περιοχή του χάρτη και ένα στη δεξιά περιοχή. Με αυτόν τον τρόπο μειώνεται σημαντικά το συνολικό κόστος μετακίνησης.

ΣΤΑΔΙΟ Δ

Σε αυτό το στάδιο μελετήθηκε η γενική μορφή του προβλήματος χωροθέτησης αεροδρομίων. Δίνονται η πόλεις με γνωστές συντεταγμένες και πληθυσμούς p_i ενώ ζητείται ο προσδιορισμός των βέλτιστων θέσεων m αεροδρομίων έτσι ώστε να ελαχιστοποιείται το συνολικό κόστος των κατοίκων.

Σε αντίθεση με τα προηγούμενα στάδια, όπου εξετάστηκαν ειδικές περιπτώσεις ενός ή δύο αεροδρομίων, στο στάδιο αυτό το πλήθος των πόλεων και των αεροδρομίων είναι μεταβλητό.

Για την επίλυση του προβλήματος χρησιμοποιήθηκαν δύο αλγόριθμοι:

1. PSO
2. Genetic Algorithm

Στόχος ήταν η σύγκριση των δύο μεθόδων ως προς την ποιότητα της λύσης και τον χρόνο εκτέλεσης.

PSO:

Ο συγκεκριμένος αλγόριθμος είναι ένας πληθυσμιακός αλγόριθμος βελτιστοποίησης. Κάθε υποψήφια λύση αναπαρίσταται από ένα σωματίδιο. Τα σωματίδια κινούνται στον χώρο αναζήτησης και προσπαθούν να βρουν την καλύτερη δυνατή λύση.

Τα πλεονεκτήματα της PSO είναι ότι είναι απλή στην υλοποίηση, έχει μικρό αριθμό παραμέτρων, γρήγορη σύγκλιση, δεν απαιτεί παραγώγους και είναι κατάλληλη για μη γραμμικά προβλήματα.

Genetic Algorithm:

Ο συγκεκριμένος αλγόριθμος είναι μεταευρετικός. Κάθε πιθανή λύση αναπαρίσταται ως χρωμόσωμα. Ένας πληθυσμός χρωμοσωμάτων εξελίσσεται μέσω διαδικασιών selection, crossover και mutation. Κατά τη διάρκεια των γενεών οι καλύτερες λύσεις επιβιώνουν και μεταφέρουν τα χαρακτηριστικά τους στις επόμενες γενιές.

Selection:

Επιλέγονται τα καλύτερα άτομα του πληθυσμού με βάση την τιμή της αντικειμενικής συνάρτησης. Συγκεκριμένα το selection γίνεται με tournament

style όπου διαλέγουμε τυχαία ένα sample από τους απογόνους και παίρνουμε τους 2 καλύτερους γονείς μέσα σε αυτό το sample.

Ωστόσο έχουμε υλοποιήσει και μια “greedy” εκδοχή του genetic algorithm στην οποία παίρνουμε μόνο τους 2 καλύτερους γονείς και δεν βρήκαμε σημαντικές διαφορές για το πρόβλημα που επιλύουμε.

Crossover:

Δύο γονείς ανταλλάσσουν τμήματα του χρωμοσώματος τους ώστε να δημιουργηθούν νέες λύσεις. Στη περίπτωση μας, οι απόγονοι που δημιουργούνται παίρνουν coordinates x , y έναν τυχαίο αριθμό μεταξύ των coordinates των γονέων. Αυτό από μόνο του είναι αρκετά περιοριστικό και για τον λόγο αυτό χρησιμοποιούμε και *mutain*.

Mutation:

Μικρές τυχαίες μεταβολές εισάγονται στις συντεταγμένες των αεροδρομίων. Αυτό βοηθά στην αποφυγή παγίδων σε τοπικά ελάχιστα.

Τα πλεονεκτήματα είναι ότι έχει ισχυρή παγκόσμια αναζήτηση, μικρή πιθανότητα παγίδευσης σε τοπικά ελάχιστα, είναι κατάλληλο για προβλήματα μεγάλης διάστασης και έχει μεγάλη ευελιξία στην αναπαράσταση λύσεων.

Για κάθε εκτέλεση αρχικά δημιουργήθηκαν τυχαίες πόλεις σε δισδιάστατο χώρο στην συνέχεια σε κάθε πόλη ανατέθηκε τυχαίος πληθυσμός έπειτα εκτελέστηκε ο αλγόριθμος PSO και ο Genetic Algorithm για τα ίδια δεδομένα και τέλος καταγράφηκαν το τελικό κόστος, ο χρόνος εκτέλεσης και οι τελικές θέσεις των αεροδρομίων. Η σύγκριση έγινε χρησιμοποιώντας την ίδια συνάρτηση κόστους και το ίδιο σύνολο πόλεων ώστε τα αποτελέσματα να είναι άμεσα συγκρίσιμα.

Παρατηρήθηκε ότι και οι δύο αλγόριθμοι κατάφεραν να βρουν πολύ καλές λύσεις. Ο PSO παρουσίασε ταχύτερη σύγκλιση, καθώς τα σωματίδια εκμεταλλεύονται άμεσα την πληροφορία της καλύτερης λύσης του πληθυσμού. Ο Genetic Algorithm αφιερώνει περισσότερο χρόνο στην εξερεύνηση του χώρου αναζήτησης μέσω των διαδικασιών crossover και mutation. Εξαιτίας αυτού συχνά απαιτεί περισσότερες επαναλήψεις, αλλά έχει μεγαλύτερη πιθανότητα να αποφύγει τοπικά ελάχιστα. Στις περισσότερες εκτελέσεις ο PSO εμφάνισε μικρότερο χρόνο εκτέλεσης ενώ ο Genetic Algorithm παρήγαγε λύσεις παρόμοιας ή ελαφρώς καλύτερης ποιότητας.

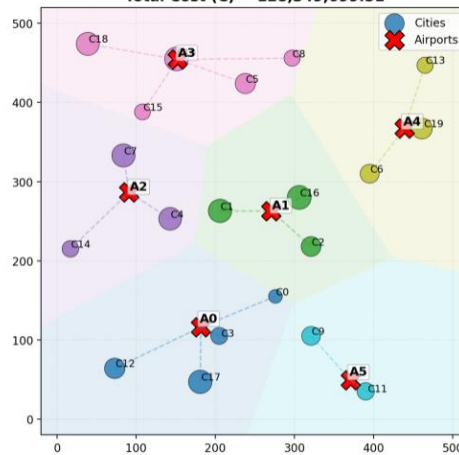
Συμπερασματικά στο στάδιο Δ υλοποιήθηκε η γενική μορφή του προβλήματος χωροθέτησης αεροδρομίων με χρήση δύο αλγορίθμων, του PSO και του Genetic Algorithm. Και οι δύο κατάφεραν να εντοπίσουν θέσεις αεροδρομίων που μειώνουν σημαντικά το συνολικό κόστος μετακίνησης του πληθυσμού. Ο PSO παρουσίασε ταχύτερη σύγκλιση και μικρότερο υπολογιστικό κόστος ενώ ο Genetic Algorithm προσέφερε ισχυρότερη εξερεύνηση του χώρου λύσεων.

Τελικά ο PSO αποτελεί ιδιαίτερα αποδοτική επιλογή για το συγκεκριμένο πρόβλημα ενώ ο Genetic Algorithm λειτουργεί ως μια αξιόπιστη εναλλακτική μέθοδο που μπορεί σε κάποιες περιπτώσεις να οδηγήσει σε ακόμη καλύτερες λύσεις.

Παρακάτω φαίνονται οι υλοποιήσεις της PSO και του Genetic Algorithm για 6 αεροδρομια καθώς και η σύγκρισή τους:

Interactive Optimization (PSO)

Total Cost (C) = 128,349,099.51

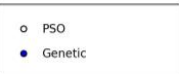
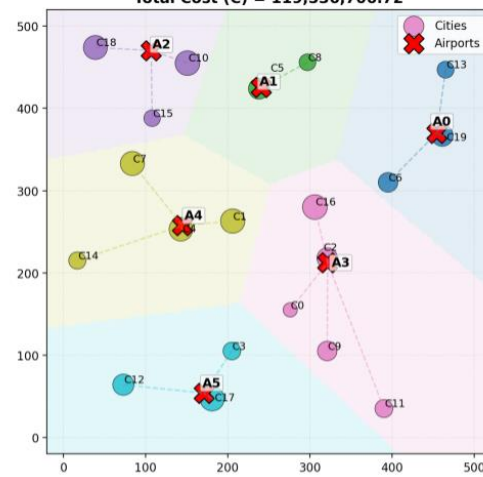


Airports (m) 6

Optimize!

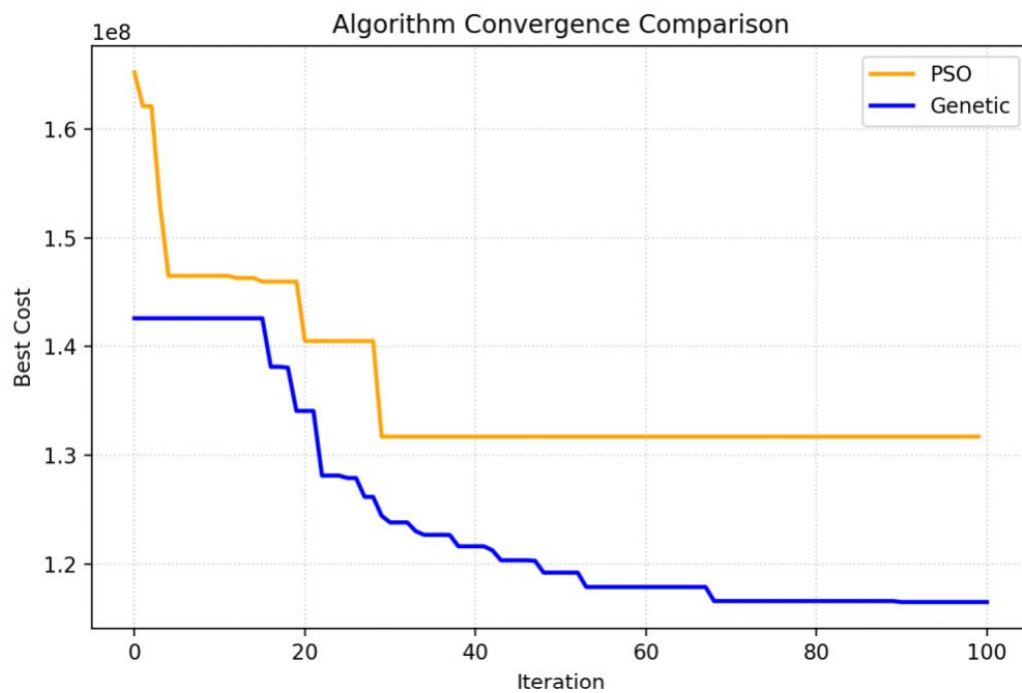
Interactive Optimization (Genetic)

Total Cost (C) = 119,536,706.72



Airports (m) 6

Optimize!



Visualization

1. Αναλυτική Επεξήγηση του visualization.py

Για να μην γράφουμε τις ίδιες γραμμές κώδικα Matplotlib σε κάθε αρχείο ξεχωριστά, μαζέψαμε όλη τη λογική σχεδίασης στο αρχείο [visualization.py](#). Το υποσύστημα αυτό πλέον περιλαμβάνει **δύο εξειδικευμένες συναρτήσεις**: τη στατική σχεδίαση ([plot_airport_system](#)) και το διαδραστικό παράθυρο ([plot_interactive_dashboard](#)).

Τα βασικά χαρακτηριστικά και οι λειτουργίες του κώδικα οπτικοποίησης:

- **Βρίσκει το κοντινότερο αεροδρόμιο:** Για κάθε πόλη υπολογίζει την απόσταση από όλα τα διαθέσιμα αεροδρόμια και την αναθέτει αυτόματα σε αυτό που είναι πιο κοντά της.
- **Υπολογίζει το συνολικό κόστος:** Πολλαπλασιάζει την απόσταση κάθε πόλης με τον πληθυσμό της και βγάζει το τελικό Κόστος (**C**) του δικτύου, το οποίο τυπώνεται δυναμικά στον τίτλο.
- **Μεγαλώνει τους κύκλους ανάλογα με τον πληθυσμό:** Οι πόλεις σχεδιάζονται ως κύκλοι, με το μέγεθός τους να μεγαλώνει ανάλογα με το πόσοι άνθρωποι μένουν εκεί, ώστε να ξεχωρίζουν αμέσως τα μεγάλα αστικά κέντρα.
- **Γεωμετρικά Σύνορα Αεροδρομίων (Voronoi Background):** Χρησιμοποιώντας ένα Grid με τη NumPy, χωρίζουμε τον χάρτη σε χιλιάδες μικρά σημεία, βρίσκουμε σε ποιο αεροδρόμιο ανήκει το καθένα και βάφουμε το φόντο με ένα ημιδιαφανές χρώμα. Έτσι, δημιουργούνται γεωμετρικά σύνορα που δείχνουν τα όρια κάθε αεροδρομίου (Voronoi Territories).
- **Ξεχωρίζει τις περιοχές με χρώματα:** Κάθε αεροδρόμιο (κόκκινο "X") παίρνει ένα δικό του χρώμα από μια έτοιμη παλέτα ([tab10](#)). Οι πόλεις, οι γραμμές σύνδεσης και το φόντο της περιοχής τους βάφονται με το ίδιο ακριβώς χρώμα.
- **Διαδραστικό Dashboard (Sliders & Κουμπιά):** Μέσω της συνάρτησης [plot_interactive_dashboard](#), ανοίγει ένα ζωντανό παράθυρο. Ο χρήστης μπορεί να αλλάξει τον αριθμό των αεροδρομίων με ένα Slider, να διαλέξει αλγόριθμο (PSO ή Γενετικό) από ένα μενού και πατώντας το κουμπί **Optimize!**, ο χάρτης και τα σύνορα στο φόντο ανανεώνονται live στην οθόνη. Επειδή ο υπολογισμός των αλγορίθμων μπορεί να πάρει λίγο χρόνο, το κουμπί αλλάζει αμέσως το κείμενό του σε "Thinking..." μόλις πατηθεί, αναγκάζοντας τα γραφικά να ανανεωθούν πριν ξεκινήσει ο βαρύς υπολογισμός, και επανέρχεται σε "Optimize!" μόλις εμφανιστεί η λύση.

- **Καθαρές Ετικέτες (Labels):** Βάζει ονόματα σε πόλεις (C0, C1...) και αεροδρόμια (A0, A1...). Για να μην πέφτουν τα γράμματα πάνω στα σύμβολα, βάλαμε μια μικρή μετατόπιση και ένα λευκό ημιδιαφανές κουτάκι (background box) πίσω από το όνομα των αεροδρομίων, οπότε διαβάζονται πεντακάθαρα.

2. Πώς εφαρμόζονται οι συναρτήσεις σε κάθε στάδιο (Parts A, B, C, D)

Ανάλογα με τις ανάγκες κάθε σταδίου, χρησιμοποιούμε τη σωστή συνάρτηση για να παίρνουμε είτε στατικά screenshots για την έκθεση, είτε live προσομοίωση για την παρουσίαση:

- **Στάδιο Α (main_A.py):** Χρησιμοποιούμε τη στατική συνάρτηση `plot_airport_system`. Επειδή δεν υπάρχει πληθυσμός, η συνάρτηση δίνει αυτόματα το ίδιο σταθερό μέγεθος σε όλες τις πόλεις. Το 1 αεροδρόμιο σχεδιάζεται στο κέντρο του δικού του ενιαίου έγχρωμου συνόρου.
- **Στάδιο Β (main_B.py):** Χρησιμοποιούμε τη στατική συνάρτηση `plot_airport_system`. Πριν την καλέσουμε, ενώνουμε τις συντεταγμένες με τον πληθυσμό σε έναν πίνακα 3 στηλών (`np.c_[cities, populations]`). Η συνάρτηση διαβάζει τον πληθυσμό, αλλάζει το μέγεθος των κύκλων και σχεδιάζει το φόντο.
- **Στάδιο Γ (main_C.py):** Χρησιμοποιούμε τη στατική συνάρτηση `plot_airport_system`. Περνάμε τα 2 αεροδρόμια που βρήκε ο Γενετικός αλγόριθμος. Ο χάρτης αυτόματα χωρίζεται σε δύο μεγάλες, έγχρωμες γεωμετρικές ζώνες.
- **Στάδιο Δ - Μεμονωμένα Αρχεία (main_D_PSO.py & main_D_Genetic.py):** Στα δύο αυτά αρχεία χρησιμοποιούμε τη στατική συνάρτηση `plot_airport_system`. Μας επιτρέπει να τρέξουμε τον κάθε αλγόριθμο ξεχωριστά με 100 iterations, να δούμε την αντίστοιχη καμπύλη σύγκλισης και να πάρουμε πεντακάθαρα, σταθερά screenshots των συνόρων για το report της εργασίας μας.
- **Στάδιο Δ - Σύγκριση (main_D_comparison.py):** Χρησιμοποιούμε τη διαδραστική συνάρτηση `plot_interactive_dashboard`. Η συνάρτηση αυτή αποτελεί το κεντρικό σημείο αξιολόγησης, καθώς εκτελεί την άμεση σύγκριση των δύο προσεγγίσεων (PSO και Γενετικού Αλγορίθμου) κάτω από τις ίδιες ακριβώς συνθήκες. Ανοίγει ένα

δυναμικό περιβάλλον όπου υπάρχει ένα **Slider** για να αλλάξει ο αριθμός των αεροδρομίων (1 έως 8) και τα **Radio Buttons** για να αλλάξει ο αλγόριθμος.

3. Οι αλλαγές στο αρχείο δημιουργίας πόλεων ([make_cities.py](#))

Στο Στάδιο Δ, η εκφώνηση ζητάει τη δημιουργία τυχαίων οικιστικών δικτύων (πόλεων). Όταν όμως οι πόλεις έμπαιναν εντελώς τυχαία στον χάρτη, πολλές φορές έπεφταν σχεδόν η μία πάνω στην άλλη. Για να το λύσουμε, αλλάξαμε τη συνάρτηση `make_cities` προσθέτοντας έναν **έλεγχο ελάχιστης απόστασης** (`min_distance=60`):

Τώρα, κάθε φορά που το πρόγραμμα πάει να γεννήσει μια νέα τυχαία πόλη, μετράει την απόστασή της από όλες τις άλλες πόλεις που έχουν ήδη φτιαχτεί. Αν δει ότι η νέα πόλη πέφτει πολύ κοντά σε κάποια υπάρχουσα, απορρίπτει αμέσως τη θέση και ξαναδοκιμάζει σε άλλη τυχαία συντεταγμένη. Έτσι οι πόλεις εμφανίζονται ρεαλιστικά στον χώρο.

GitHub

1. Setup του Αποθετηρίου (Repository)

Για την ομαλή συνεργασία των 4 ατόμων της ομάδας, το πρώτο βήμα που κάναμε ήταν να στήσουμε ένα κεντρικό αποθετήριο (Repository) στο GitHub. Με αυτόν τον τρόπο, όλος ο πηγαίος κώδικας, τα δεδομένα και οι εκθέσεις μας βρίσκονταν συγκεντρωμένα σε έναν ασφαλή χώρο στο cloud, προσβάσιμο από όλα τα μέλη ανά πάσα στιγμή.

Για τη διαχείριση του κώδικα επιλέξαμε να χρησιμοποιήσουμε την εφαρμογή **GitHub Desktop**. Η επιλογή αυτή αποδείχθηκε εξαιρετικά βοηθητική, καθώς μας επέτρεψε να συγχρονίζουμε τα αρχεία μας, να βλέπουμε τις αλλαγές που έκανε ο καθένας και να ανεβάζουμε τη δουλειά μας με απλά κλικ, χωρίς να χρειάζεται να μπλέκουμε με πολύπλοκες εντολές στο τερματικό.

2. Ποή Εργασίας με Κλάδους (Git Branches)

Για να μην επικαλύπτει ο ένας τη δουλειά του άλλου και για να αποφύγουμε το χάος, εφαρμόσαμε τη στρατηγική των **παράλληλων κλάδων (Branches)**.

- **Κεντρικός Κλάδος (main):** Κρατήσαμε τον κεντρικό κλάδο αυστηρά καθαρό. Εκεί βρισκόταν μόνο ο τελικός, λειτουργικός κώδικας που ξέραμε σίγουρα ότι τρέχει χωρίς σφάλματα.
- **Κλάδοι Εργασίας (Feature Branches):** Κάθε μέλος της ομάδας, όταν ξεκινούσε να δουλεύει στο δικό του κομμάτι (π.χ. στο Visualization ή στον PSO), δημιουργούσε έναν δικό του, προσωπικό κλάδο (π.χ. `feature/map-visualization`). Ο καθένας έγραφε και δοκίμαζε τον κώδικά του, χωρίς να επηρεάζει τους υπόλοιπους.
- **Pull Requests & Merge:** Όταν ένα κομμάτι του project ολοκληρωνόταν (όπως για παράδειγμα η γενική συνάρτηση οπτικοποίησης), ο υπεύθυνος φοιτητής άνοιγε ένα Pull Request στο GitHub. Τα υπόλοιπα μέλη της ομάδας έριχναν μια ματιά στον κώδικα για επιβεβαίωση και, αφού συμφωνούσαμε, γινόταν η συγχώνευση (Merge) των αλλαγών στον κεντρικό κλάδο `main`.